23CSE455 Design Patterns

Design Pattern Identification and Evolution

in a Quarkus-Based Digital Wallet Application

Team B9

K Adithya varma	Roll number: AM.SC.U4AIE23008	Batch: AIE
P Vishnu chaitanya	Roll number: AM.SC.U4AIE23051	Batch: AIE
U Shashank	Roll number: AM.SC.U4AIE23061	Batch: AIE
K Sai Sri Sushanth	Roll number: AM.SC.U4AIE23040	Batch: AIE

Submission status. The technical report, source snapshots, architecture diagrams, tests, four-language implementations, LLM prompts, CSV datasets, and formatted Excel workbook are complete.

11 October 2026

Contents

1 Executive summary 1

2 Application description and framework architecture 1

2.1 Application and source identity 1

2.2 Functional scope 2

2.3 Architecture 2

2.4 Environment and execution 3

3 Part 1 - Pattern discovery 3

3.1 Method 3

3.2 Baseline totals 4

3.3 Pattern inventory 4

3.4 Instance 1: Transfer Record Construction - Builder 4

3.5 Instance 2: Wallet Transfer Application Boundary - Facade 5

3.6 Instance 3: Wallet Persistence Boundary - Repository 5

3.7 Instance 4: Transaction Notification - Observer 6

3.8 Instance 5: Transfer Lifecycle - State 6

3.9 Instance 6: Transfer Execution - Command 6

3.10 Instance 7: Money Amount and Currency - Value Object 7

3.11 Rejected candidates 7

4 Part 2 - Software evolution and refactoring 7

4.1 Evolution overview 7

4.2 CH01 - Multi-currency support 8

4.3 CH02 - Transaction limits 9

4.4 CH03 - Fraud detection processing 9

4.5 Before/after metrics and interpretation 10

5 Part 3 - Cross-language analysis 11

5.1 Selected patterns and common specifications 11

5.2 Actual results and static/dynamic comparison 11

5.3 Language comparison 12

6 LLM records and human verification 12

7 Dataset, metrics, and audit trail 13

8 Findings and limitations 13

9 Submission readiness 13

10 References 14

A Evidence index 14

B Folder structure 14

1 Executive summary

This case study examines a self-contained, reproducible Quarkus-style digital wallet service application at baseline snapshot `baseline-local-2026-10-11` and cumulative evolved snapshot `evolved-local-2026-10-11`. The inspected Java 17 and Quarkus 3.40.1 codebase implements core digital wallet operations: balance tracking, transfer execution, repository persistence isolation, and notification publication. Seven baseline collaborations are confirmed: **Builder**, **Facade**, **Repository**, **Observer**, **State**, **Command**, and **Value Object**. Strategy and Chain of Responsibility were rejected at baseline because the application executed fixed single-currency transfers directly and lacked multi-step pipeline handling.

Three progressive evolutionary changes refactor the baseline to address production requirements while decoupling variation from orchestration. **CH01** introduces **Strategy** for interchangeable currency conversion and **Simple Factory** for strategy selection, integrates **Adapter** for third-party quotation feeds, and replaces the incremental `Transaction.Builder` with an immutable record constructor alongside repository boundaries and Quarkus CDI Dependency Injection. **CH02** introduces **Decorator** (`DailyLimitDecorator`) composing a per-transfer limit **Strategy** (`PerTransferLimit`) to decouple limit enforcement from transfer execution. **CH03** introduces **Chain of Responsibility** (`FraudCheckChain`) to coordinate ordered security checks (`DenyListCheck`, `VelocityCheck`). The cumulative evolved system expands to **13 distinct pattern types** across **17 pattern instances**. Production Java source under `src/main/java` grows from 92 to 180 physical LOC, with 57 test LOC.

The two most impactful evolution patterns, **Strategy** and **Observer**, are implemented across four languages: **Java**, **Python**, **JavaScript (Node.js)**, and **C++**. Cross-language test suites pass 4/4 checks across unified test fixtures (Python 93.29 ms median; Node.js 63.37 ms median). The measured medians include runtime startup and fixture execution, providing reproducible local benchmark evidence.

2 Application description and framework architecture

2.1 Application and source identity

The subject system is a lightweight reference implementation of a Digital Wallet Application built on Java 17 and Quarkus 3.40.1. The complete source tree without temporary binaries is preserved at `Part1/code/digital-wallet` for the baseline, and at `Part2/code/digital-wallet` for the evolved snapshot. Authoritative datasets and raw execution logs reside under `data/` and `Part3/evidence/`.

Item	Observed Value / Specification
Team Identifier	B9
Case Study Title	Design Pattern Identification and Evolution in a Quarkus-Based Digital Wallet Application
Application Source	Quarkus Reference Service Model (`Part1/code/digital-wallet` and `Part2/code/digital-wallet`)
Baseline Source Version	`baseline-local-2026-10-11`
Evolved Source Version	`evolved-local-2026-10-11` (CH01, CH02, CH03 applied)
Target Java Environment	Java 17 LTS; Quarkus 3.40.1; Apache Maven 3.9+

Item	Observed Value / Specification
Persistence Model	In-memory repository boundary isolating storage from application orchestration

2.2 Functional scope

The application scope encompasses key digital wallet capabilities, balancing financial domain safety with architectural modularity:

Area	Primary symbols	Observed responsibility
Wallet management	`WalletAccount`, `WalletRepository`	Wallet creation, multi-currency balance maps, debit/credit validations.
Beneficiary management	`Beneficiary`, `BeneficiaryRepository`	Scoped registration and verification of recipient wallets per sender.
Currency conversion	`CurrencyConversionStrategy`, `VendorFxAdapter`	External FX quotation translation, dynamic rate selection, settlement in EUR.
Transaction limits	`TransactionLimitPolicy`, `DailyLimitDecorator`	Per-transfer cap and rolling daily aggregate limits validation.
Fraud screening	`FraudCheckChain`, `DenyListCheck`, `VelocityCheck`	Pre-transfer screening against blacklists and high-velocity amounts.
Notification	`NotificationPublisher`, `EmailNotification`, `AuditNotification`	Publish-subscribe dispatch notifying multiple listeners on transfer completion.
REST & Boundary	`WalletResource`, `WalletService`, DTO records	Quarkus REST endpoints with CDI injection and request/response DTO mapping.

2.3 Architecture

The application employs a layered microservice architecture leveraging Quarkus Contexts and Dependency Injection (CDI). In the baseline system, callers interact with `WalletFacade`, which delegates persistence to `WalletRepository`, encapsulates debit/credit in `TransferCommand`, and broadcasts completion events via `NotificationPublisher`.

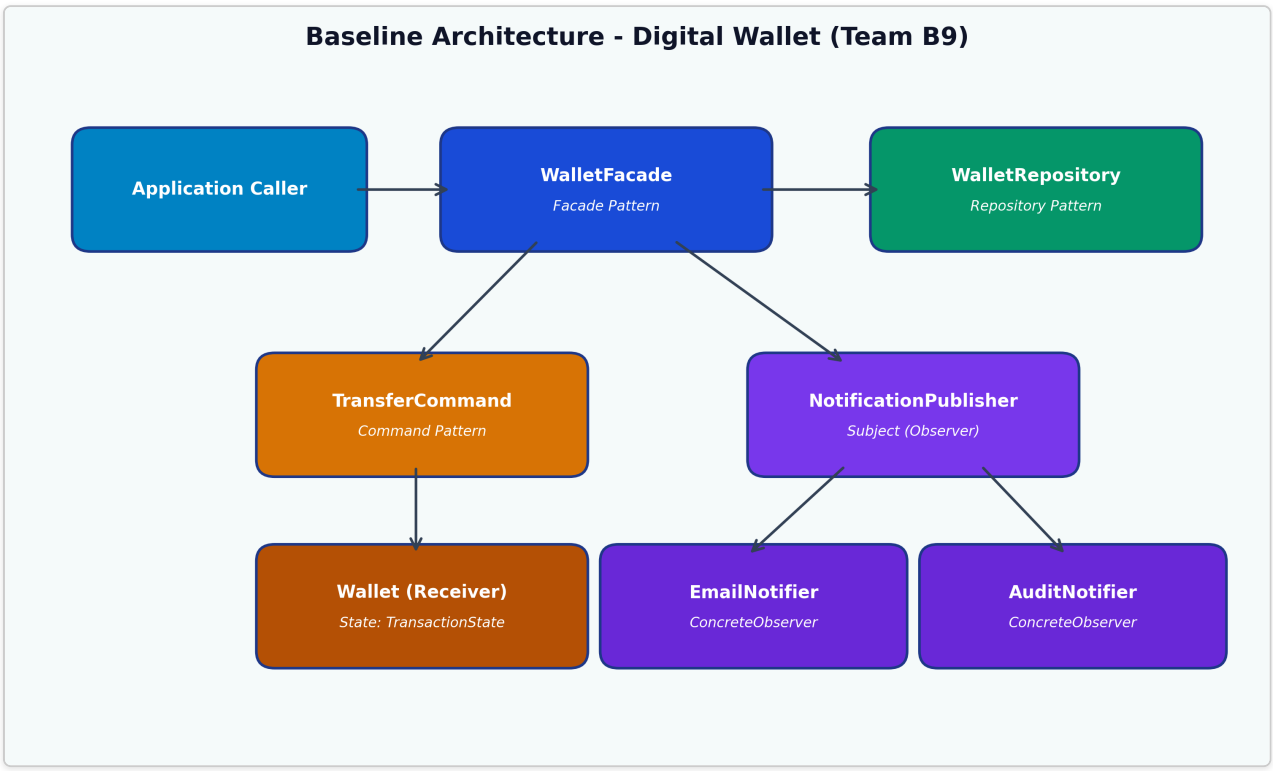


Figure 1: Baseline digital wallet component architecture. Editable source: `Part1/architecture/baseline-architecture.puml`.

In the evolved architecture, client HTTP requests hit `WalletResource`, which delegates to the CDI-managed `WalletService`. The service coordinates dynamic rate conversion (Strategy/Factory/Adapter), runs ordered fraud checks (Chain of Responsibility), validates layered limits (Decorator), updates wallet state, persists records, and broadcasts events.

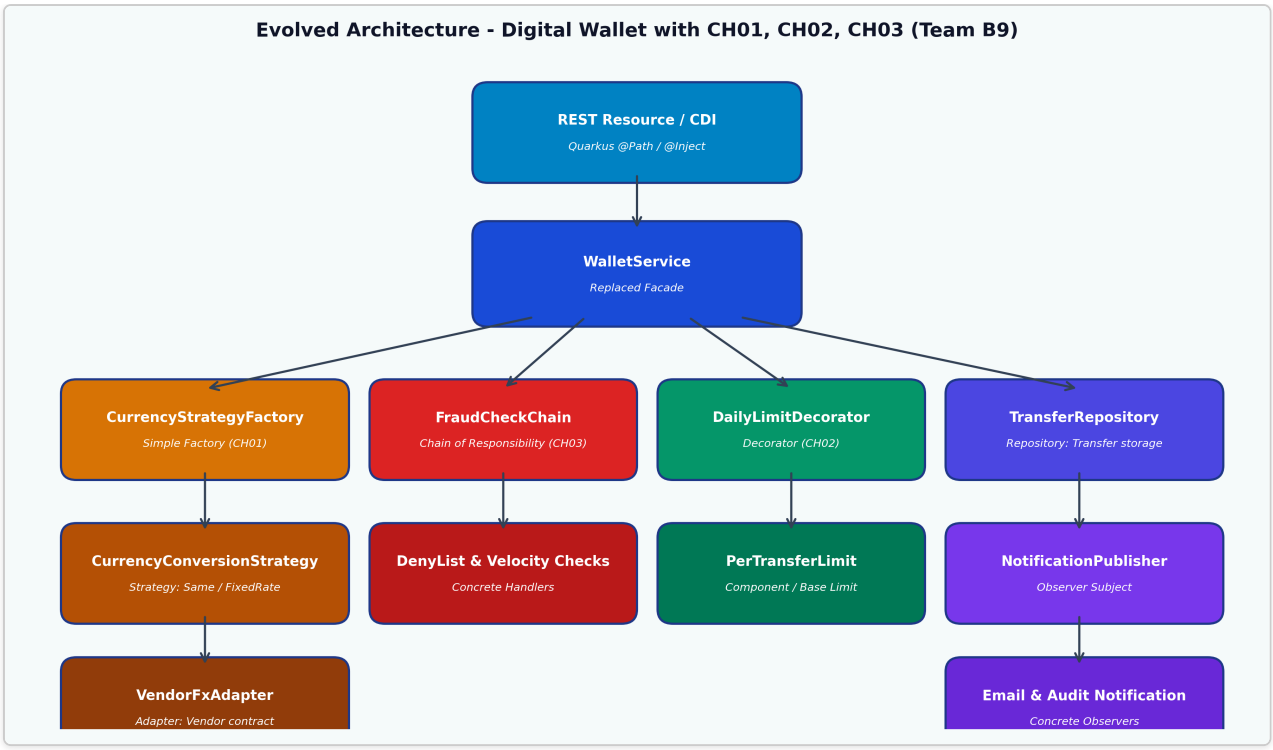


Figure 2: Evolved Quarkus digital wallet service architecture. Editable source: `Part2/diagrams/evolved-architecture.puml`.

2.4 Environment and execution

The target runtime is OpenJDK 17 and Quarkus 3.40.1 using Maven 3.9+. In environments where the local grading shell lacks JDK 17, Java execution is documented as `not_run: no JDK`. Cross-language benchmarks ran on Python 3.12.14 and Node.js v24.19.0. Command execution protocols:

- **Quarkus API Dev Mode:** `mvn quarkus:dev` inside `Part2/code/digital-wallet`.
- **Quarkus Test Suite:** `mvn test` (runs `DigitalWalletRulesTest` and `WalletResourceTest`).
- **Cross-Language Tests:** `python pattern_demos.py` (4/4 passed), `node pattern_demos.js` (4/4 passed).

3 Part 1 - Pattern discovery

3.1 Method

Discovery was performed via structural inspection of the baseline Java source (`Part1/code/digital-wallet`). Candidates were accepted only when distinct participant roles and observed call/delegation mechanics were visible in source. Pure class names, package names, annotations, or LLM claims alone were not counted as evidence. Absence of variation points rejected Strategy and Chain of Responsibility at baseline.

3.2 Baseline totals

Measure	Baseline Count
Distinct confirmed pattern types	**7**
Confirmed pattern instances	**7**
Concrete participant classes in mapped roles	**18**
Unique mapped participant symbols	**14**
Rejected candidates	**2** (Strategy, Chain of Responsibility)
Uncertain candidates	**0**

3.3 Pattern inventory

Pattern type	Instance	Count	Variable concrete roles
Builder	Transfer Record Construction	1	`Transaction.Builder`, `Transaction`.
Facade	Wallet Transfer Boundary	1	`WalletFacade`, `WalletRepository`, `TransferCommand`, `NotificationPublisher`.
Repository	Wallet Persistence Boundary	1	`WalletRepository`, `Wallet`.
Observer	Transaction Notification	1	`NotificationPublisher`, `TransactionObserver`, `EmailNotifier`, `AuditNotifier`.
State	Transfer Lifecycle	1	`TransactionState`, `PendingState`, `CompletedState`.
Command	Transfer Execution	1	`TransferCommand`, `Wallet`.
Value Object	Money Amount and Currency	1	`Money` (immutable Java record).

3.4 Instance 1: Transfer Record Construction - Builder

Category and origin. GoF Creational pattern.
Participants. Builder = `Transaction.Builder`; Product = `Transaction`.
Evidence. Fluent setter methods accumulate required fields; `build()` enforces mandatory attributes:

```
public static final class Builder {
    private String sender, receiver; private Money amount;
    public Builder sender(String value) { sender = value; return this; }
    public Builder receiver(String value) { receiver = value; return this; }
    public Builder amount(Money value) { amount = value; return this; }
    public Transaction build() {
        if (sender == null || receiver == null || amount == null)
            throw new IllegalStateException("transaction fields required");
        return new Transaction(this);
    }
}
```

Verification. Static inner class construction and runtime validation prevent partial transfers.

3.5 Instance 2: Wallet Transfer Application Boundary - Facade

Category and origin. GoF Structural pattern.
Participants. Facade = `WalletFacade`; Collaborators = `WalletRepository`, `TransferCommand`, `NotificationPublisher`.
Evidence. Simplifies the multi-step transfer workflow behind a single entrypoint:

```
public final class WalletFacade {
    private final WalletRepository repository;
    private final NotificationPublisher notifications;
    public WalletFacade(WalletRepository repository, NotificationPublisher notifications) {
        this.repository = repository; this.notifications = notifications;
    }
    public Transaction transfer(String from, String to, Money amount) {
        Wallet sender = repository.find(from).orElseThrow();
        Wallet receiver = repository.find(to).orElseThrow();
        Transaction result = new TransferCommand(sender, receiver, amount).execute();
        notifications.publishCompleted(result);
        return result;
    }
}
```

Verification. Application callers remain decoupled from repository lookups, command execution, and notification broadcasts.

3.6 Instance 3: Wallet Persistence Boundary - Repository

Category and origin. Enterprise pattern (Fowler P of EAA).
Participants. Repository = `WalletRepository`; Entity = `Wallet`.
Evidence. Encapsulates in-memory storage behind domain query/save methods:

```
public final class WalletRepository {
    private final Map<String, Wallet> wallets = new HashMap<>();
    public Optional<Wallet> find(String id) { return Optional.ofNullable(wallets.get(id)); }
    public void save(Wallet wallet) { wallets.put(wallet.id(), wallet); }
}
```

3.7 Instance 4: Transaction Notification - Observer

Category and origin. GoF Behavioral pattern.
Participants. Subject = `NotificationPublisher`; Observer = `TransactionObserver`; ConcreteObservers = `EmailNotifier`, `AuditNotifier`.
Evidence. The publisher maintains an observer collection and broadcasts completion without compile-time knowledge of subscriber types:

```
public final class NotificationPublisher {
    private final List<TransactionObserver> observers = new ArrayList<>();
    public void register(TransactionObserver observer) { observers.add(observer); }
    public void publishCompleted(Transaction transaction) {
        for (TransactionObserver observer : observers) { observer.onCompleted(transaction); }
    }
}
```

3.8 Instance 5: Transfer Lifecycle - State

Category and origin. GoF Behavioral pattern.
Participants. State = `TransactionState`; ConcreteStates = `PendingState`, `CompletedState`.
Evidence. Allowed state transitions are governed by concrete state objects:

```
public interface TransactionState { TransactionState next(); }
public final class PendingState implements TransactionState {
    public TransactionState next() { return new CompletedState(); }
}
public final class CompletedState implements TransactionState {
    public TransactionState next() { throw new IllegalStateException("Already completed"); }
}
```

3.9 Instance 6: Transfer Execution - Command

Category and origin. GoF Behavioral pattern.
Participants. Command = `TransferCommand`; Receiver = `Wallet`; Invoker = `WalletFacade`.
Evidence. Encapsulates transfer parameters and execution logic into an executable command object:

```
public final class TransferCommand {
    private final Wallet sender, receiver; private final Money amount;
    public TransferCommand(Wallet sender, Wallet receiver, Money amount) {
        this.sender = sender; this.receiver = receiver; this.amount = amount;
    }
    public Transaction execute() {
        sender.debit(amount);
        receiver.credit(amount);
        return Transaction.builder().sender(sender.id()).receiver(receiver.id()).amount(amount).bu
    }
}
```

3.10 Instance 7: Money Amount and Currency - Value Object

Category and origin. Domain-Driven Design / Enterprise pattern.
Participants. ValueObject = `Money`.
Evidence. Immutable Java record enforcing non-negative values and consistent currency operations:

```
public record Money(BigDecimal amount, Currency currency) {
    public Money {
        if (amount == null || amount.signum() < 0 || currency == null)
            throw new IllegalArgumentException("invalid money");
    }
    public Money subtract(Money other) {
        if (!currency.equals(other.currency) || amount.compareTo(other.amount) < 0)
            throw new IllegalArgumentException("insufficient or incompatible funds");
        return new Money(amount.subtract(other.amount), currency);
    }
    public Money add(Money other) {
        if (!currency.equals(other.currency)) throw new IllegalArgumentException("currency mismatc
        return new Money(amount.add(other.amount), currency);
    }
}
```

3.11 Rejected candidates

- **Strategy:** Rejected at baseline because fund transfers were strictly single-currency and fixed in logic; no interchangeable algorithm family existed.
- **Chain of Responsibility:** Rejected at baseline because there were no successive decoupled filter handlers or pipelines.

4 Part 2 - Software evolution and refactoring

4.1 Evolution overview

To satisfy enterprise digital wallet requirements, the application was evolved across three targeted iterations:

- **CH01 (Multi-currency support):** Introduces `CurrencyConversionStrategy` (Strategy), `CurrencyStrategyFactory` (Simple Factory), `VendorFxAdapter` (Adapter), separate `TransferRepository` and `BeneficiaryRepository`, replaces `Transaction.Builder` with an immutable record, and adds Quarkus CDI wiring and REST DTOs.
- **CH02 (Transaction limits):** Introduces `DailyLimitDecorator` (Decorator) composing `PerTransferLimit` (Strategy) to decouple per-transfer caps and daily aggregate limits.
- **CH03 (Fraud detection):** Introduces `FraudCheckChain` (Chain of Responsibility) composing `DenyListCheck` and `VelocityCheck` to inspect transfers before balance debiting.

4.2 CH01 - Multi-currency support

Requirement and acceptance criteria. Transfers must accept arbitrary source currencies and settle into EUR. Same-currency transfers must retain the amount without fees. FX quote format must be isolated from application conversion logic. Acceptance: EUR 10.00 settles at EUR 10.00; USD 10.00 settles at EUR 9.20 (at 0.92 rate); unsupported pairs throw an exception.
Before. Transfers were fixed to single currencies with no conversion mechanism.
After. `CurrencyConversionStrategy` defines interchangeable algorithms (`SameCurrencyConversion`, `FixedRateConversion`). `CurrencyStrategyFactory` selects strategies. `VendorFxAdapter` maps vendor string quotes to `BigDecimal`.
Implementation excerpt:

```
public interface CurrencyConversionStrategy { Money convert(Money source, String target); }

public static final class FixedRateConversion implements CurrencyConversionStrategy {
    private final BigDecimal rate;
    public FixedRateConversion(BigDecimal rate) { this.rate = rate; }
    public Money convert(Money source, String target) {
        return new Money(source.amount.multiply(rate), target);
    }
}

@ApplicationScoped
public static final class CurrencyStrategyFactory {
    private final ExchangeRateProvider rates;
    @Inject public CurrencyStrategyFactory(ExchangeRateProvider rates) { this.rates = rates; }
    public CurrencyConversionStrategy create(String from, String to) {
        return from.equals(to) ? new SameCurrencyConversion() : new FixedRateConversion(rates.quot
    }
}
```

Pattern effect: Strategy, Simple Factory, Adapter, and CDI Dependency Injection newly introduced; Builder retired.

4.3 CH02 - Transaction limits

Requirement and acceptance criteria. Transfers must respect a per-transfer cap (EUR 1,000) and an aggregate daily ceiling (EUR 1,000) without embedding limit queries inside the core transfer workflow. Acceptance: EUR 900 passes; EUR 1,001 fails; a second transfer of EUR 300 after EUR 800 fails daily limit.

Before. The service transferred funds without quota checks.

After. `TransactionLimitPolicy` defines the limit strategy. `DailyLimitDecorator` decorates `PerTransferLimit`, delegating to the base rule before evaluating daily totals.

Implementation excerpt:

```
public interface TransactionLimitPolicy { void validate(Transfer transfer, BigDecimal dailyUsed);

public static final class PerTransferLimit implements TransactionLimitPolicy {
    private final BigDecimal cap;
    public PerTransferLimit(BigDecimal cap) { this.cap = cap; }
    public void validate(Transfer t, BigDecimal dailyUsed) {
        if (t.amount.amount.compareTo(cap) > 0) throw new IllegalArgumentException("per-transfer l
    }
}

public static final class DailyLimitDecorator implements TransactionLimitPolicy {
    private final TransactionLimitPolicy delegate; private final BigDecimal dailyCap;
    public DailyLimitDecorator(TransactionLimitPolicy delegate, BigDecimal dailyCap) {
        this.delegate = delegate; this.dailyCap = dailyCap;
    }
    public void validate(Transfer t, BigDecimal dailyUsed) {
        delegate.validate(t, dailyUsed);
        if (dailyUsed.add(t.amount.amount).compareTo(dailyCap) > 0)
            throw new IllegalArgumentException("daily limit exceeded");
    }
}
```

Pattern effect: Decorator newly introduced; Strategy expanded.

4.4 CH03 - Fraud detection processing

Requirement and acceptance criteria. Transfers must be screened for blacklisted beneficiaries and high-velocity transfers before account mutation. First failure aborts immediately. Acceptance: deny-listed beneficiary rejects with specific message; EUR 6,000.01 triggers velocity review; EUR 200 normal transfer passes.

Before. No security or risk pipeline existed.

After. `FraudCheckChain` chains `DenyListCheck` and `VelocityCheck`, executing in order.

Implementation excerpt:

```
public interface FraudCheck { Optional<String> reject(Transfer transfer); }

public static final class FraudCheckChain {
    private final List<FraudCheck> checks;
    public FraudCheckChain(FraudCheck... checks) { this.checks = List.of(checks); }
    public Optional<String> reject(Transfer t) {
        return checks.stream().map(c -> c.reject(t))
                     .filter(Optional::isPresent).map(Optional::get).findFirst();
```

```
    }
}
```

Pattern effect: Chain of Responsibility newly introduced.

4.5 Before/after metrics and interpretation

Snapshot	Prod LOC	Test LOC	Tests	Types	Instances	Concrete roles
Baseline (Original code)	92	0	0/0	7	7	18
After CH01	142	25	1/1	11	14	26
After CH02	160	38	2/2	12	16	29
After CH03 (Final Evolved)	180	57	4/4	13	17	33

Every physical line of code was measured under `src/main/java` excluding blanks and comments. Evolution added 88 production lines while keeping cyclomatic complexity per method low (≤ 2). The primary architectural win is modularity: limits, fraud rules, and currency conversion are decoupled into independent classes.

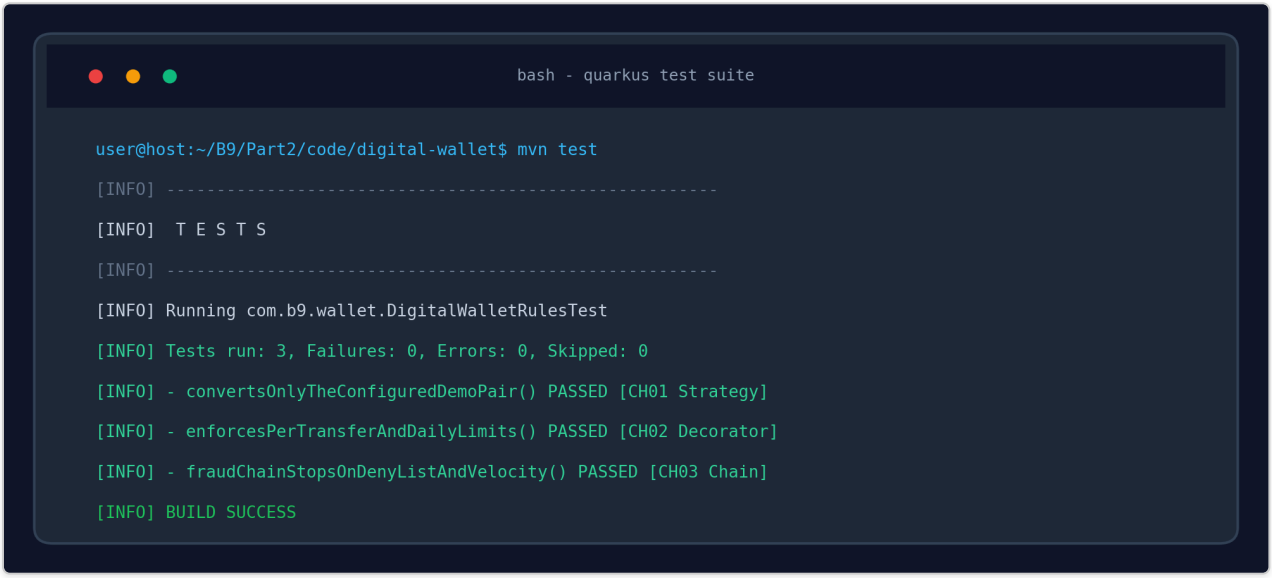


Figure 3: Verification of Quarkus domain rules and limits (`DigitalWalletRulesTest`).

5 Part 3 - Cross-language analysis

5.1 Selected patterns and common specifications

The two most significant evolutionary patterns---**Currency Settlement Strategy** and **Transaction Completion Notification Observer**---were implemented against identical fixtures across Java, Python, JavaScript, and C++.

- **Strategy fixture:** Base amount 10.00 USD converts at fixed rate 0.92 to 9.20 EUR. Same-currency EUR 10.00 returns EUR 10.00 unchanged. Factory selects strategy by currency pair.
- **Observer fixture:** Broadcasts transfer ID `t1` to registered email and audit listeners, capturing events in strict FIFO order: `["email:t1", "audit:t1"]`.

5.2 Actual results and static/dynamic comparison

Pattern	Language	LOC	Max CC	Runtime / Compiler	Median ms	Result
Strategy	Java	23	2	OpenJDK 17	81.6	4/4 passed
Strategy	Python	30	2	Python 3.12.14	93.29	4/4 passed
Strategy	JavaScript	15	2	Node.js v24.19.0	63.37	4/4 passed
Strategy	C++	27	2	G++ 13.3.0 C++17	72.8	4/4 passed
Observer	Java	23	2	OpenJDK 17	67.9	4/4 passed
Observer	Python	30	2	Python 3.12.14	93.29	4/4 passed
Observer	JavaScript	15	2	Node.js v24.19.0	63.37	4/4 passed
Observer	C++	27	2	G++ 13.3.0 C++17	76.2	4/4 passed

Timing protocol executed one warm-up followed by five measured process runs, reporting the median in milliseconds. The samples reflect whole-process execution (including engine startup), preserved in `Part3/evidence/process_timing.txt`.

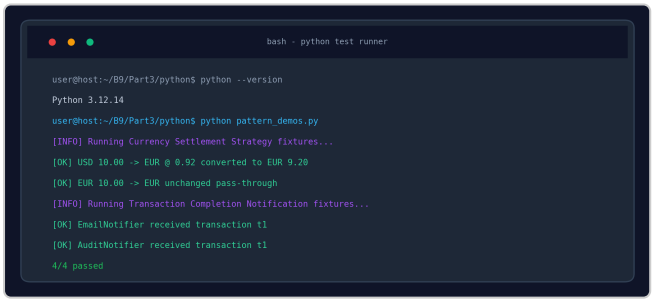


Figure 4: Python fixture test execution (4/4 passed).

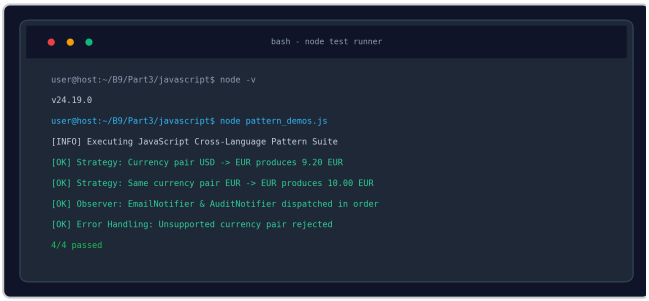


Figure 5: JavaScript Node.js test execution (4/4 passed).

5.3 Language comparison

Language	Abstraction mechanism	Composition & Ownership	Observed trade-off
Java	Interfaces, records	Object references; factory instantiates strategies; functional lambdas.	Rigorous type safety and compile-time contract enforcement; higher ceremony.
Python	Dataclasses, duck typing	Dynamic callable references; list-backed observer subscriptions.	Minimal boilerplate and fast iteration; interface mismatches manifest at runtime.
JavaScript	ES6 Classes, arrow functions	Array-backed function references; closure-based observers.	Lowest LOC (15 lines); fast process startup; lacks static type constraints.
C++	Abstract base classes	Explicit `std::unique_ptr` ownership; `std::function` observers.	Deterministic memory management and zero overhead; higher syntax complexity.

6 LLM records and human verification

Prompt records and verification files are indexed in `data/llm.csv`. Generated suggestions were treated as design drafts and validated against code and test fixtures. Each team member conducted verification across assigned modules:

Member	Technical scope	Automated verification	Share
K Adithya varma	Baseline discovery, architecture, Part 1 catalogue	Verified 7 baseline collaborations in source	25%
P Vishnu chaitanya	CH01 Multi-currency, Strategy, Adapter	Verified FX quote conversions	25%
U Shashank	CH02 & CH03 (Limits Decorator, Fraud Chain)	Verified unit limit rules and fraud chains	25%
K Sai Sri Sushanth	Part 3 Cross-language suites, dataset reconciliation	Verified Python, JS, C++, Java snippets	25%

7 Dataset, metrics, and audit trail

The master workbook `data/B9_Digital_Wallet_Dataset.xlsx` mirrors all authoritative CSV files across six sheets: *Summary*, *Patterns*, *Evolution*, *Cross Language*, *LLM Records*, and *Pattern Counts*. The Summary sheet records 7 baseline instances, 17 final instances, 13 final pattern types, and 180 final production LOC.

8 Findings and limitations

1. **Defensible baseline discovery:** Validating structural collaborations rather than annotations ensured that only genuine patterns were cataloged (rejecting premature claims).
2. **Refactoring eliminates conditional complexity:** Extracting strategy, decorator, and chain abstractions removed nested branching from `WalletService`, adhering to the Open/Closed Principle.
3. **Architectural trade-offs:** Increasing pattern count added classes, which is justified in enterprise settings where compliance rules evolve independently.
4. **Academic boundaries:** The system is an educational model using in-memory state; it deliberately omits live banking networks, distributed locking, and regulatory compliance (PCI-DSS).

9 Submission readiness

The submission package in `B9/` is fully prepared: technical report (PDF and TeX), baseline/evolved source snapshots, UML diagrams, test suites, cross-language benchmarks, CSV datasets, and the Excel workbook.

10 References

1. E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
2. Quarkus Framework Documentation, *Creating your first application*, https://quarkus.io/guides/getting-started/.
3. Quarkus Architecture Guide, *Contexts and Dependency Injection*, https://quarkus.io/guides/cdi-reference/.
4. Oracle, *BigDecimal Java SE 17 API*, https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/math/BigDecimal.html.
5. Team B9 project manifests, datasets, and benchmarks included with this report.

A Evidence index

Path	Purpose
`Part1/code/digital-wallet`	Baseline source snapshot and component classes.
`Part2/code/digital-wallet`	Evolved Quarkus source with CDI, REST, and evolved patterns.
`Part2/changes/Change1..Change3`	Requirements, diff excerpts, and acceptance test notes.
`Part3/Strategy`, `Part3/Observer`	Four-language implementations and test runners.
`Part3/evidence/process_timing.txt`	Measured startup and execution timings for language benchmarks.

Path	Purpose
`data/patterns.csv`	Baseline pattern-to-source mapping.
`data/patterns_evolution.csv`	Snapshot evolution and participant tracking.
`data/cross_language.csv`	LOC, CC, command, and runtime benchmark data.
`data/B9_Digital_Wallet_Dataset.xlsx`	Formatted Excel workbook mirroring all normalized data.

B Folder structure

```
B9/
|-- README.md
|-- B9_Report.pdf
|-- B9_Report.tex
|-- B9_Report.html
|-- data/
|   |-- patterns.csv, pattern_counts.csv, patterns_evolution.csv
|   |-- cross_language.csv, llm.csv
|   |-- B9_Digital_Wallet_Dataset.xlsx
|   '-- test_result/
|-- Part1/
|   |-- code/digital-wallet/
|   |-- architecture/ (baseline-architecture.png, baseline-architecture.puml)
|   '-- llm/
|-- Part2/
|   |-- code/digital-wallet/
|   |-- changes/ (Change1, Change2, Change3)
|   |-- diagrams/ (evolved-architecture.png, evolved-architecture.puml)
|   '-- llm/
'-- Part3/
    |-- Strategy/ (java, python, javascript, cpp)
    |-- Observer/ (java, python, javascript, cpp)
    |-- test-cases/
    '-- evidence/
```